

Tab 1

WEEK - 3

SELECT Command:

Examples on retrieving data from a single table using

- i) SELECT statement
- ii) SELECT statement with where clause (Comparison Operators, AND, OR, NOT, IN, BETWEEN, LIKE)
- iii) ORDER BY clause (sort by column name)
- iv) LIMIT clause

Case 1:

i. SELECT:

The SELECT statement in MySQL is used to retrieve data from one or more tables in a database. The basic syntax of a SELECT statement is as follows:

Syntax:

SELECT column1, column2, ...FROM table_name;

```
insert into student(name,rno) values("surya",25);
insert into student(name,rno) values("raju",225),("karthik",244),("naveesh",255);
select name from student;
```

output:

Result Grid		Filter Rows:
	name	
▶	surya	
	raju	
	raju	
	karthik	
	naveesh	

Case 2:

i. SELECT statement with where clause

SELECT statement with where clause syntax

SELECT column1, column2, ...FROM table_name WHERE condition;

example:

```
insert into student(name,rno) values("raju",225),("karthik",244),("naveesh",255);
select name from student where rno=225;
```

output:

Result Grid		Filter Row
	name	
▶	raju	

ii. SELECT statement with WHERE clause using Comparison Operators.

In MySQL, and many other SQL databases, you can use various comparison operators in the **WHERE** clause to filter rows based on specific conditions. Here are some common comparison operators:

- =: Equal to
- != or <>: Not equal to
- <: Less than
- <=: Less than or equal to
- >: Greater than
- >=: Greater than or equal to

Equal to (=):

Used to check if two expressions are equal.

SELECT * FROM table_name WHERE column_name = value;

example:

```
insert into student(name,rno) values("raju",225),("karthik",244),("naveesh",255);
select name from student where rno=225;
```

Output:

Result Grid		Filter Row
	name	
▶	raju	

Not equal to (!= or <>):

Used to check if two expressions are not equal.

SELECT * FROM table_name WHERE column_name != value;

or

SELECT * FROM table_name WHERE column_name <> value;

example:

```
select * from student where rno != 225;
```

output:

Result Grid   Filter Rows:			
	name	rno	salary
▶	surya	25	20000
	karthikeya	226	15000
	sreenu	229	10000
	karthik	244	1200
	naveesh	255	5000
	karthik	244	1200
	naveesh	255	5000

Greater than (>):

Used to check if one expression is greater than another.

SELECT * FROM table_name WHERE column_name > value;

example:

```
select * from student where salary > 5000;
```

example:

Result Grid   Filter Row			
	name	rno	salary
▶	surya	25	20000
	karthikeya	226	15000
	sreenu	229	10000

Less than (<):

Used to check if one expression is less than another.

SELECT * FROM table_name WHERE column_name < value;

Greater than or equal to (>=):

Used to check if one expression is greater than or equal to another.

SELECT * FROM table_name WHERE column_name >= value;

Less than or equal to (<=):

Used to check if one expression is less than or equal to another.

SELECT * FROM table_name WHERE column_name <= value;

example:

```
select * from student where salary <= 15000;
```

output:

Result Grid			
	name	rno	salary
▶	karthikeya	226	15000
	sreenu	229	10000
	karthik	244	12000
	naveesh	255	5000

IS NULL:

Used to check if a column contains a NULL value.

SELECT * FROM table_name WHERE column_name IS NULL;

example:

```
insert into student(name,rno,salary) values("raju",225,25000),("karthik",244,12000),("navee",250,10000);
select * from student where salary is null;
```

output:

Result Grid			
	name	rno	salary

IS NOT NULL:

Used to check if a column does not contain a NULL value.

SELECT * FROM table_name WHERE column_name IS NOT NULL;

example:

```
select * from student where salary is not null;
```

output:

Result Grid			
	name	rno	salary
▶	surya	25	20000
	karthikeya		15000
	sreenu	229	10000
	raju	225	25000
	karthik	244	12000
	naveesh	255	5000

iii. SELECT statement with WHERE clause using AND , OR ,NOT.

a. WHERE clause with AND.

SELECT column-names

FROM table-name

WHERE condition1 AND condition2

example:

```
select * from student where salary >=20000 and rno =225 ;
```

output:

Result Grid			
	name	rno	salary
▶	raju	225	25000

b. WHERE clause with OR

SELECT column1, column2, ...

FROM table_name

WHERE condition1 OR condition2;

example:

```
select * from student where salary >=20000 or rno =225 ;
```

output:

Result Grid			
	name	rno	salary
▶	surya	25	20000
	raju	225	25000
	karthik	244	32000

c. WHERE clause with NOT

SELECT column1, column2, ...

FROM table_name

WHERE NOT condition;

example:

```
select * from student where not salary >=20000 ;
```

output:

Result Grid			
	name	rno	salary
▶	karthikeya	226	15000
	sreenu	229	10000
	naveesh	255	5000

d. SELECT statement with WHERE Clause Using IN operator

The **IN** operator in a **WHERE** clause is used to specify a range for multiple values in a column. It allows you to match a value against a set of values. Here's the syntax for a **SELECT** statement with the **IN** operator:

SELECT column1, column2, ...FROM table_name WHERE column_name IN (value1, value2, ...);

Example:

SELECT employee_id, first_name, last_name, department

FROM employees

WHERE department IN ('Sales', 'Marketing');

```
select * from employ where branch in("sales");
```

Result Grid			
	empid	name	branch
▶	1	surya	sales
	3	karthik	sales

e. SELECT statement with WHERE clause using BETWEEN Operator

The **BETWEEN** operator in a **WHERE** clause is used to filter rows based on a range of values. It's commonly used with numeric, date, or timestamp columns. Here's the syntax for a **SELECT** statement with the **BETWEEN** operator:

Syntax:

SELECT column1, column2, ...

FROM table_name

WHERE column_name BETWEEN value1 AND value2;

Example:

SELECT product_id, product_name, price

FROM products

WHERE price BETWEEN 50 AND 100;

```
select * from employ where prices between 20 and 50;
```

Result Grid				
	empid	name	branch	prices
▶	2	raju	marketing	30
	3	karthik	sales	20
	4	naveesh	marketing	40

f. **SELECT** statement with **WHERE** Clause using **LIKE** operator.

The **LIKE** operator in a **WHERE** clause is used to search for a specified pattern in a column. It is often used with wildcard characters such as **%** (percent) and **_** (underscore). Here's the syntax for a **SELECT** statement with a **WHERE** clause using the **LIKE** operator:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE column_name LIKE pattern;
```

Example:

```
SELECT product_id, product_name
```

```
FROM products
```

```
WHERE product_name LIKE '_Laptop_';
```

```
select * from employ where name like '%a';
```

Result Grid				
Filter Rows:				
	empid	name	branch	prices
▶	1	surya	sales	10
	2	raja	marketing	30

Case 3:

iii) **ORDER BY** clause (sort by column name)

ORDER BY clause

The **ORDER BY** clause in a **SELECT** statement is used to sort the result set based on one or more columns. Here's the basic syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ...;
```

Example:

```
SELECT first_name, last_name, salary
```

```
FROM employees
```

```
ORDER BY salary DESC;
```

```
select * from employ order by prices desc;
```

Result Grid   Filter Rows:				
	empid	name	branch	prices
▶	4	naveesh	marketting	40
	2	raja	marketing	30
	3	karthik	sales	20
	1	surya	sales	10

Case 4:

The **LIMIT** clause in a **SELECT** statement is used to restrict the number of rows returned by a query. It is often used in combination with the **ORDER BY** clause to get a specific subset of rows based on a certain order. Here's the basic syntax:

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ...
```

```
LIMIT number_of_rows;
```

Example:

```
SELECT first_name, last_name, salary
```

```
FROM employees
```

```
ORDER BY salary DESC
```

```
select * from employ order by prices desc limit 3;
```

Result Grid   Filter Rows:				
	empid	name	branch	prices
▶	1	surya	sales	50
	4	naveesh	marketting	40
	2	raja	marketing	30

Tab 2

